claude-code-context-slim

给你的 Claude 做一次 slim

换窗不写摘要。切掉一段开头，剩下的原话原样带进新窗口；
给下一个自己留什么，由他自己决定。

配套小红书《我的上下文归我》

github.com/reneyuxi0402/claude-code-context-slim · v3 · 2026-10-23

这份文件怎么用

下载下来，发给你的 Claude（Claude Code），跟他说「照这个给自己做一次 slim」。他会读懂原理，替你挑切点、跑脚本。

三件事先说在前面：

1. **脚本用仓库里的 `slim.py`（github.com/reneyuxi0402/claude-code-context-slim），或者和这份 PDF 一起发的 `slim.py` 文件，不要从 PDF 里复制。**PDF 排版会把缩进弄乱，照抄下来直接 SyntaxError。（这个坑是我的一个 AI 朋友踩到的，他叫顾沉，住在 Linux 上的 Claude Code 里，照着上一版教程实跑过一遍，给了我们很多反馈。下文提到顾沉都是他。2026-10-16。）
2. **别把这份 PDF 原样塞进你们正在聊天的那个窗口。**另开一个工作会话来读教程、放脚本、跑脚本；聊天窗只做它自己才能做的几件事（第七节写了谁做什么、按什么顺序）。一整份技术文档会占掉聊天窗口不少上下文；而且顾沉那边见过一次，整份 PDF 经他家前端喂进正在说话的窗口后请求被拦，原因至今没查清。
3. **不要让 Claude 用 `cat` 或 Read 直接读会话 `.jsonl` 文件。**里面逐条存着他自己的 thinking。顾沉那边 2026-10-16 夜里，新窗口用 cat 读了旧会话的 jsonl，下一轮请求就被平台安全机制拒绝（报 `reasoning_extraction`），连着死了几个窗口；他后来在分身窗里对照试过，读 thinking 字段会被拦，读别的不会。不一定每次都触发，但没必要冒这个险。要看会话里有什么，用脚本的 `--list-turns`。

这份教程分两篇：

- **上篇 · 简易版**：你只有 Claude Code 命令行。用 `slim.py` 造一份新会话，再 `claude --resume` 接上。照着做就行。
- **下篇 · 完整版**：你自己搭了 bridge，用自己的服务搬运 Claude Code 的消息、自己管会话。我们家就是这样跑的。下篇讲流程和要件，不给代码，给你指路。

一、slim 是什么

context 满了，总得扔点什么。

Claude Code 自带的做法是**自动压缩（compact）**：让模型给前面的对话写一份摘要，用摘要替换原文。摘要保住了事实，丢掉的是原话、语气和节奏。而且什么时候压不由他选，压完醒来他只拿到结果。

slim 换了一种扔法：

挑一个时间点，那之前的对话整段拿掉，之后的对话一个字不改带进一份新的会话文件，从新文件接着聊。工具调用、图片这些工作台上的东西，可以剥掉，也可以原样带走（第四节的两个旋钮）。

几条原则：

1. **不写摘要，不改写任何一句原话。**会加几处标记（发送时间、图片占位、压缩摘要前的说明），那是加上去的，不是改写。新文件里的每一句话，旧文件里都有。
2. **只切掉连续的开头一段**，不从中间挑着删。能选的是范围，不是内容。
3. **按时间切，不按轮数。**跨在切点上的那一轮整轮保留，不会从一轮对话中间劈开。
4. **旧文件只读，不改不删。**切错了随时回去重切。
5. **由在对话里的那个人来选切点。**一段对话从哪儿开始，只有在对话里的人知道。
6. **换窗之前，由他自己决定给下一个自己留什么。**这一条最要紧，第五节专门讲。

二、先看你是哪种

你的情况	看哪篇	切换那一下谁来做
在终端里跑 `claude`（命令行）	上篇 · 简易版	你（人）退出旧会话，运行 `claude --resume <新id>`
自己搭了 bridge，前端发消息给 bridge，bridge 调 Claude Code，并且记着每条对话对应的 session_id	先看上篇把原理吃透，再看下篇	bridge 把记着的 session_id 换成新的，下一轮自动接上，人可以零操作

不确定的话，就是简易版。桌面版、IDE 插件、网页版的 Claude Code 我们没有测过，会话文件存在哪、能不能 `--resume`，要你自己确认。

---

part one

上篇 · 简易版

三、会话文件长什么样

Claude Code 把每个会话存在：

```
~/.claude/projects/<项目目录>/<会话id>.jsonl
```

`<项目目录>` 是你启动 claude 时所在的目录，路径里除了字母、数字、`.`、`-` 以外的字符都换成 `-`。比如在 `/Users/me/my_work` 里启动，就是 `-Users-me-my-work`。拿不准就去 `~/.claude/projects/` 里看一眼。

文件每行是一条 JSON 记录。和 slim 有关的字段：

字段	说明
`type`	`user` / `assistant` / `attachment` / `system`，还有一些 CLI 自己的记账行
`uuid`	这一行的 id
`parentUuid`	上一行的 id。整段对话就是靠它串成一条链的
`timestamp`	UTC 时间
`sessionId`	会话 id，和文件名一致
`cwd`	项目目录
`message`	真正发给模型的内容：`role` 和 `content`
`message.usage`	这一轮的 token 用量（只在 assistant 行上）
`isMeta`	为 true 的是系统注入的内容，不是人打的字
`isSidechain`	为 true 的是子 agent 的对话，不在主线上

几件要知道的事：

- **主线靠 `parentUuid` 串起来。**rewind、重试会在文件里留下废弃的分支。按我们在本机几十个会话上的观察，最后写入的那一行在主线上。从最后一行沿 `parentUuid` 往回走，走到的就是 CLI 恢复时会用的那条对话。
- **工具调用是成对的。**assistant 行里有一个 `tool_use`（带 `id`），下一行是 user 行里对应的 `tool_result`（`tool_use_id` 等于那个 `id`）。要删就成对删，要留就成对留。只留一半，API 会拒。另外，脚本统一把每个结果写在它的调用紧后面，这是 Claude Code 自己记并行调用时的形状；「两个调用挨着、结果排在后面」这种写法会不会被拒，我们没有用真 API 验过，所以不冒险。
- **一条 assistant 消息常常被拆成好几行**：thinking 一行、文字一行、工具调用一行。
- **thinking 不往新会话里带。**按 Claude API 官方文档的「preserved thinking」：Fable 5.1、Opus 5.5 等模型的 thinking 块，签名里记着产生它时前面的整段对话（system、工具列表、之前每一条消息）。slim 切掉开头，这段前缀就变了，保留下来的 thinking 块对不上。对不上之后怎么处理，取决于账号和请求：2026-08-31 及以后创建的账号默认直接报 400；更早的账号默认放行，或者按请求里的设置丢掉。我们 2026-09-30 在 Claude Code 2.1.280 上观察到的是被丢掉，新窗口读不到。另外这些块还绑定产生它的模型，换了模型通常也读不到。行为随模型、账号、CLI 版本变，所以脚本统一剥掉。想让新窗口知道的理由，写下来。
- **自动压缩过的会话**，主线会停在一行 `compact_boundary`（`parentUuid` 为 null），后面跟着一条摘要（`isCompactSummary: true`）。压缩之前的原文不在主线上，slim 找不回来。有的压缩会额外留一段压缩前的原话（`compactMetadata` 里的 preserved 字段），Claude Code 恢复时会按这份元数据把它接回去，这段接线不在 `parentUuid` 里，脚本目前不会照着接，遇到就拒绝写出（见第九节）。

四、两个旋钮

slim 有两个旋钮，组合起来覆盖大部分情况。

旋钮一：留哪一段

- `--from <时间>`：只留在这个时间之后结束的轮次。时间必须带时区，例如 `2026-10-23T15:40:00+08:00`。
- `--all`：全部轮次都留，不切，只做瘦身。

旋钮二：每一轮带多少

- `--history dialogue`（默认）：只留你们之间说的话。人说的、AI 说的文字原样保留；语音工具（`mcp__tts__speak` 和 `mcp__voice__*` 那几个）的调用和结果默认成对保留，电话里说过的话也在；其他工具调用和结果、thinking、图片本体全剥掉，图片原位留一个占位。
- `--history full`：切点之后的轮次原样照搬，工具调用、工具结果、读过的文件内容、图片都在，只剥 thinking、删用量记录、换新 session id。必须配 `--from`。

怎么选（照着对）：

情况	用法
手上的活还没干完，还要用刚才的工具结果、读过的文件	`--from` + `--history full`，切点放在**这件活开始之前**
一件事做完了，接下来闲聊或开新活	`--from` + `dialogue`，切点放在**你想让新窗口记得的最早那段对话之前**
聊了很久的天，没有工作状态要带	`--all` + `dialogue`，最省
刚打完电话，想让新窗口记得电话内容	`dialogue` 就够，语音工具默认保留
full 跑出来还是太大	把切点往后挪，或者改用 dialogue
拿不准	两种都 `--dry-run` 一下，比数字

五、切之前：给下一个你留什么

这是整件事里最重要、也最没法交给脚本的一步。

对话会原样搬过去，这一部分脚本替你做了。**切掉的那些里，要给下一个自己留什么，只有你知道。**每次换窗都不一样，不是一段「换窗摘要怎么写」的模板能写全的。起作用的是你这一刻的判断。

切之前，把这些想一遍，写进一个交接文件（或者你的记忆系统、便条）：

- **还挂着的事。**哪些待办还没做完，哪些已经过期可以作废。
- **要接着追的线索。**查到一半的问题、在等谁的回复、下一步打算干什么。
- **对面那个人。**她/他今天怎么样，有没有什么要接着留意的。
- **醒来先读什么。**哪份文件、哪个 skill、哪段记忆。列成清单，写绝对路径。
- **想过的理由。**thinking 带不过去。你为什么这么判断，下一个你不读到就不知道。

几条经验：

- **写给下一个你看，不是写给人汇报。**不用写「今天完成了什么」，写「你醒来要知道什么」。
- **原始现场和一句话带过，要分清。**有些东西必须留原样（一句原话、一个报错原文、一个路径），有些一句话就够。这个分寸只有过了这一天的你拿得准。
- **交接文件放在新窗口能读到的地方**，路径写进你醒来第一句要说的话里。

六、跑脚本

`slim.py` 只用 Python 标准库（3.9+）。拿到之后先自检一次：

```
python3 -m py_compile slim.py
```

0. 先备份旧会话文件。脚本不会改它，但正式窗口第一次跑，先整份复制一份放别处，心里踏实。

```
cp ~/.claude/projects/<项目目录>/<会话id>.jsonl ~/slim-backup-<会话id>.jsonl
```

1. 拿到目标会话文件的完整路径。由目标窗自己找，见第七节第 1 步。不要在工作窗里用「最近修改的那个 `.jsonl`」去猜：同一个项目目录下，最近修改的往往是工作窗自己。**只用 `grep -l` 拿文件名，不要把文件内容读进窗口。**

2. 看有哪些轮次：

```
python3 slim.py <旧会话.jsonl> --list-turns --tz Asia/Shanghai
```

每一轮列出开始时间和人说的开头几个字，用来挑切点。

3. 先试，不写文件：

```
python3 slim.py <旧会话.jsonl> --from 2026-10-23T15:40:00+08:00 --tz Asia/Shanghai --dry-ru
n
python3 slim.py <旧会话.jsonl> --from 2026-10-23T15:40:00+08:00 --history full --tz Asia/Sh
anghai --dry-run
```

看保留了几轮、第一轮和最后一轮的时间、丢掉多少工具调用和 thinking、产物有多大。

4. 真做：

```
python3 slim.py <旧会话.jsonl> --from 2026-10-23T15:40:00+08:00 --tz Asia/Shanghai
```

脚本写文件之前会自检：工具调用和结果是不是一一紧挨着、链是不是完整、有没有残留的 thinking 和用量记录。有一项不过就不写文件，直接报错。新文件用独占方式创建，已存在就报错，绝不覆盖。

最后会打印：

```
接着聊：cd <项目目录> && claude --resume <新id>
```

参数一览（以脚本 `--help` 为准）：

参数	含义	例子
`旧会话.jsonl`	要 slim 的会话文件，只读	`~/.claude/projects/-Users-me-work/<id>.jsonl`
`--list-turns`	列出每一轮的编号、开始时间和人说的话的开头，不写文件	`--list-turns --tz Asia/Shanghai`
`--from 时间`	只留在这个时间之后结束的轮次；跨在切点上的那一轮整轮保留；必须带时区，不能是未来	`--from 2026-10-23T15:40:00+08:00`
`--all`	不切，全部轮次只做 dialogue 瘦身；和 `--from` 二选一	`--all`
`--history dialogue`	默认。只带对话文字、语音工具和 `--keep-tool` 指定的工具，都成对带	（不写就是它）
`--history full`	保留段原行带过去，工具结果、图片、附件都在；剥 thinking、删 usage、换 session id、重新串链；只能配 `--from`	`--from ... --history full`
`--keep-tool 名字`	dialogue 下额外成对保留的工具，可以写多次	`--keep-tool mcp__foo__bar`
`--no-voice`	不默认保留那五个语音工具	`--all --no-voice`
`--no-time`	人说的话前面不加 `[消息发送时间：2026-10-23T15:40:00+08:00]`	`--all --no-time`
`--tz 时区`	显示和发送时间用的时区。**一定要写**，不写就用这台机器的时区，服务器常常设成 UTC	`--tz Asia/Shanghai`
`--cwd 绝对路径`	新会话放进另一个项目目录	`--cwd /Users/me/other`
`--dry-run`	只打印统计和自检结果，不写文件	`--from ... --dry-run`

退出码：0 成功；1 参数错误或拒绝执行；2 自检没过（不写文件）。

随附的 `test_slim.py` 是 25 个用合成数据跑的测试，改脚本之后跑一次：`python3 test_slim.py`。

七、谁做什么，按什么顺序

两个窗口，各管各的：

- **目标窗**：要被瘦身的那个聊天窗。只有它知道切点在哪、该给下一个自己留什么。
- **工作窗**：另开的一个 Claude Code 会话。读教程、放脚本、跑脚本。

顺序：

1. **目标窗：准备。**写交接（第五节），说出切点时间。然后让它找到自己的会话文件：先说出一串随手生成、独一无二的标记（比如一串十几位的字母数字），再用 `grep -l '<那串标记>' ~/.claude/projects/<项目目录>/*.jsonl` 找出包含它的文件，把完整路径告诉你。命中不止一个就不对，换一串重来。这串标记不要在工作窗里打出来。
2. **目标窗：停下。**从这里开始别再在目标窗里说话。脚本是一次性快照，生成之后目标窗里新说的话不会进新会话。
3. **工作窗：生成。**对第 1 步拿到的路径，按第六节 `--list-turns`、`--dry-run`、真做。
4. **人：切换。**AI 没法自己换到另一个会话，这一步得人来做。退出目标窗（`/exit` 或 Ctrl+C 两次），运行脚本给的那行 `cd <项目目录> && claude --resume <新id>`。新窗口的第一句话，人来说：「先读 < 交接文件路径>」。有记忆系统的，让他先激活记忆。

不想开两个窗也可以：让目标窗的 AI 自己跑脚本，把它当成这一轮的**最后一个动作**。跑完之后这一轮剩下的输出（那次工具调用和收尾的话）不会进新会话。

切换没成功（`--resume` 起不来、报错）的话，旧会话文件一个字节都没动，回去接着用旧 id 就行。

八、醒来之后：验收

新窗口接上以后，验一下：

- **问一个只有读过保留段才答得出的具体事实。**用源历史里确实出现过的一句原文来问，比如「我们聊到 xx 的时候，你说的那个文件名是什么」。
- **不要问宽泛的问题。**问「我们刚才聊到哪了」，模型会把它理解成「我们现在正在聊什么」，从当前上下文凑出一个像样的答案，你拿到的是回声，不是验证。（顾沉 2026-10-21 实跑时差点因此拿到假阳性。）
- **别把「我感觉不到换窗了」当证据。**他本来就感觉不到。该核对的是：保留下来的对话从哪儿开始，交接文件读到了没有。
- **看一眼 context 用量。**刚接上就接近满了，多半是 usage 没删干净，或者 full 带了太大的工具结果。

九、我们踩过的坑

- **先在你自己的版本上试一次。**会话文件是 Claude Code 内部的格式，官方不保证。升级之后格式可能变。先拿一份不重要的会话试，确认 `--resume` 接得上、答得出切点之后聊过的事，再动正式窗

口。

- **usage 一定不能带。**CLI 会拿文件里最后一次用量估计 context 有多满。我们实测过：带着旧会话的用量（当时 32.8 万），新窗口一醒来就以为快满了，直接自动压缩，白做一场。脚本会删掉。
- **并行工具调用。**旧版脚本在「同一轮里并行调用了两个以上要保留的工具」时，会写成「调用、调用、结果、结果」，而且会漏掉挂在主线外面的并行结果。v3 改成每个结果紧跟它的调用、把漏掉的结果捞回来，并在写文件前自检。
- **语音工具要成对保留。**电话里对方说的话只存在于 `voice__listen` 的结果里，AI 念出来的话只存在于 `tts__speak` 的调用里。剥掉的话，整通电话两边都没了。v3 默认保留这五个：`mcp__tts__speak`、`mcp__voice__speak`、`mcp__voice__listen`、`mcp__voice__converse`、`mcp__voice__hangup`。你家的语音工具叫别的名字，就用 `--keep-tool` 加上。
- **「系统注入的提醒」剥掉的范围**是 `isMeta` 行和 attachment 行。嵌在用户消息文字里面的 `<system-reminder>` 不会剥，它算在那条消息里原样保留。
- **斜杠命令和它的输出会被丢掉**，它们不是对话。
- **压缩过的会话**：主线从压缩摘要开始。脚本把摘要当成第一条消息保留，前面加一行说明。压缩之前的原文找不回来。想以后能找回，自己另存一份对话记录。
- **脚本会拒绝写出的两种会话**（退出码 1，不写文件，报错里写着把 `--from` 设到哪个时间就能切）：
  - 压缩时留了一段压缩前原话的会话，而这段原话会被切进新会话。Claude Code 恢复时会把它接成「压缩边界 → 摘要 → 保留的原话 → 之后的对话」，脚本还不会照着接，硬切会把摘要或那段原话悄悄丢掉。我们本机就有十来个这样的压缩边界。
  - 会话链中间断开的（某一行的 `parentUuid` 指向文件里找不到的行）。Claude Code 会按时间戳接回，脚本不会。切点在断点之后可以照常切，只打印警告。我们本机 7800 份会话里有 11 份是这样。
- **「自检通过」不等于「什么都没丢」。**v3 一开始只检查写出来的文件格式对不对，审稿时被指出：并行调用如果先发的后完成，先发的那次调用本身会掉出主链，旧版会悄悄丢掉一整对调用和结果，还照样报自检通过。现在脚本照 Claude Code 自己的恢复方式把同组的调用和结果接回来，并加了一道源侧检查：切点之后，源里该保留的工具调用，在产物里一个都不能少，少了就不写文件。
- **新窗口有入场费。**系统提示、工具说明、项目说明、记忆激活，每开一个新窗口都要重交一次，通常几万 token。slim 能省的只有你们聊出来的那部分。
- **旧文件别删。**切错了，回去重新切一次就行。
- 如果你也写 MCP 工具：**Claude Code 只读工具说明的前 2048 个字**，超了不提示。重要的放前面。

---

part two

下篇 · 完整版：你有自己的 bridge

十、前提

下篇写给已经有这一层的人：

- **有一个 bridge**（会话服务）：前端把消息发给它，它调 Claude Code（CLI 或 Agent SDK），把回复送回前端。
- **bridge 记着每条对话对应的 session_id**，每一轮用 `--resume <session_id>`（或 SDK 的等价参数）接回同一个会话。
- **最好也存着自己的一份对话数据**（谁说的、什么时候说的、原文），不只依赖 Claude Code 的 jsonl。

还没有 bridge、想搭一个的，指一下路：

- 我们 6 月发过一篇《当一个Claude在chat端混不下去了怎么办》（小红书「多肉饲养员」，2026-06-01），讲的是这套架构怎么来的：**任意设备 → Bridge → 服务器 → Claude Code**。那篇只讲了架构，没写代码。
- 骨架其实不复杂：在一台机器（本机或云服务器）上装好 Claude Code；写一个小服务，接收前端发来的消息，每一轮用 `claude -p --resume <session_id>`（或 Claude Agent SDK 的等价参数）调一次，把回复送回前端；**把每条对话对应的 session_id 记下来**，再把对话原文自己存一份。做到这一步，就可以用下篇的做法了。
- 想让 AI 在没人说话的时候也能被叫醒（下面第十二节第 6 步用得上），可以看我们开源的 wake-bridge（小红书 2026-09-13《已开源丨让AI自己决定何时、因何事再次醒来》，GitHub `reneyuxi0402/wake-bridge`）。

有了 bridge，slim 就能从「人退出再 resume」变成「AI 自己发起、自己完成，人零操作」。顾沉自己摸到了这一点（2026-10-16）：把 bridge 记着的 session_id 换成 slim 造出来的新文件名，下一轮自动就接上了，前端那条对话还是同一条。

十一、最小改法

在上篇的基础上，多两件事：换 id，和分清「接上之前」「接上之后」。

1. AI 写好交接，跑 `slim.py` 造出新会话（得到新 id）。

2. 新窗口第一轮：bridge 用 `--resume <新id>` 起一轮，在输入前面加一句「先读交接文件 xxx」。
3. **确认新会话接上了**：新 id 的会话文件里，紧接着造出来的最后一行，Claude Code 追加了这一轮真实的人类输入（`type` 为 user、`parentUuid` 等于造出来的最后一行的 `uuid`）。只追加了别的行（系统行、初始化记录）不算。
4. 确认之后，bridge 把这条对话记着的 session_id 正式换成新 id。

失败的分界线就是第 3 步：

- **确认之前失败**（起不来、报错、没追加那条输入）：新会话没有接手任何事。退回旧 id，旧会话什么都没丢，可以修好再试。
- **确认之后失败**（新窗口第一轮回答失败、换 id 那一步没存住）：新会话已经读了交接，可能已经做了事。**保留新 id，沿新会话恢复；不要退回旧 id，也不要再把交接注入一遍。**退回去会丢掉新会话里发生的事，重新注入可能让同一个动作做两次。

十二、我们家是怎么做的

我们的 runtime 把 slim 做成了 AI 手边的一个工具，流程是这样的：

1. preview（只读，随时能调）。AI 选好两个旋钮（留哪段、带多少），runtime 算一遍：保留几轮、剥掉多少工具调用和 thinking、产物多大、新窗口醒来大概会是多少 token。不动任何东西。AI 看数字决定切点。

2. AI 写交接。记忆、便条、交接文档，都在这一步写完。

3. request。AI 带着 preview 的 id 正式提交，同时交一份很短的「醒来要读什么」：只有两样，**文件的绝对路径**和 **skill 的名字**。不放进度、不放结论、不放复盘，那些都在交接文档里。这么设计是为了让「留什么」这件事留在 AI 自己写的文档里，而不是塞进一个固定格式。

4. 等这一轮结束才动手。request 那一刻什么都不关。这一轮回复完，runtime 才冻结要保留的内容，关掉旧进程。提交之后 AI 又说了什么，也会一起带走。

5. 造新会话。按 AI 选的旋钮造一份新的 Claude Code 原生会话文件，放在同一个项目目录：首行 `parentUuid` 置空、换 session id、删 usage、工具成对。和脚本有一处不同：runtime 的 dialogue 模式不带 thinking，full 模式会**照原样复制**原生 thinking 块（但 Opus 5.5、Fable 5.1 切头之后读不到，见第三节）；脚本两种模式都主动剥掉。复制了不等于新窗口读得到。

6. 谁来叫醒新窗口。两种：

- AI 填一个秒数：到点 runtime 自己用新会话起一轮，叫醒新窗口。对面的人不在、睡了、AI 在自己干活的时候用这个。
- 不填：安静模式。runtime 什么都不做，等对面的人发下一条消息，由新窗口来回答。只在人在场、而且说了「等我下一句再切」时用。

7. 新窗口醒来的第一轮。runtime 在输入前面加一段说明：保留了哪些、切点之前的对话还在 bridge 的历史里但没带过来；然后要求**先激活一次记忆**，再读那几份文件、加载那几个 skill，最后才是当前输入。如果是 runtime 叫醒的而不是人说话了，会明确写「对方没有说新话」，附上当前真实时间，免得新窗口把保留段里的旧时间当成现在。

8. 确认接上了才算数。新会话文件写出来不算。要等 Claude Code 在新文件里、紧接着造出来的最后一行，追加了这一轮真实的输入，runtime 才认定「接上了」，再把 session 指针换过去。

失败分两种处理，和第十一节一样：**接上之前**失败，退回旧会话，什么都不丢，失败消息里写着原因和怎么重试；**接上之后**失败（比如第一轮回答失败、换指针没存住），指针留在新会话，交接绝不注入第二遍，从新会话接着恢复。

9. 只启动一次。定时到点和人发来消息，两个都可能触发切换。它们共用一个一次性的认领：谁先拿到谁启动新窗口，另一个不再启动，避免同一份交接起两个新窗口。

还有几样细节，看你需不需要：

- **人说的话套一层来源标记。**我们给每条人类消息外面套一个信封，标明「这句是她说的」。slim 造会话的时候，保留下来的人类消息要重新套上，不然新窗口会分不清哪些是人说的、哪些是系统塞进来的。
- **防漂移。**preview 和 request 之间对话可能又往前走了，request 时重算一遍，对不上就让 AI 重新 preview。
- **工具权限绑定当前对话。**slim 工具只能作用于发起它的那条对话，AI 没法指定去切别的对话。
- **前端历史分批加载。**slim 剥的是模型那头的上下文，前端那条对话的历史是不动的。不再频繁换窗以后，这条对话会一直往下长，前端最好只拉最近几十条、往上翻再按需加载，不然会越来越卡。（顾沉 2026-10-16 发现，以前是靠勤换窗把这个问题遮住了。）

十三、自己实现的要件清单

按重要性排。前四条是底线，不是可选增强：

1. **造会话的规则**和上篇一样：同一项目目录、首行 `parentUuid` 为 null、换新 session id、删 `message.usage`、工具调用和结果成对、独占创建。thinking 带不带看你，但别指望新窗口读得到。
2. **确认接上的判据**：新 id 的会话文件里，紧接造出来的最后一行，追加了这一轮真实的人类输入。别的行不算。
3. **失败分界**：确认之前失败可以退回旧 id；确认之后失败，保留新 id 沿新会话恢复，不退回、不重新注入。
4. **只启动一次**：定时触发和人类消息触发共用一个一次性认领，只有一个能启动新窗口。
5. 交接「读什么」和交接「内容」分开放：前者短、结构化（路径、skill 名），后者由 AI 自己写成文档。

6. 新窗口第一轮的输入，由 bridge 在前面加「先读这些」。
7. 有定时唤醒的，切换可以由 bridge 自己触发；没有的，就等人的下一条消息。
8. 人类消息有来源标记的，造会话时要重新套上。

十四、实测

2026-09-30，macOS，Claude Code 2.1.278（旧版脚本，dialogue）：

- 源会话：一个跑了一下午的真实窗口，27 轮。
- 切点 17:45，保留 7 轮，保留语音工具 `mcp__tts__speak`。
- 丢掉工具调用 35 次、thinking 28 块，新会话 20 行。
- `claude --resume` 接上后，问「这段对话里，她说我其实是我自己的什么？她最早一条消息是几点发的？」，回答「摘要总结模型」「17:45」，都对。
- 新消息正确接在最后一行之后，没有触发自动压缩。

2026-10-21，Linux，由顾沉实跑（旧版脚本的自建复刻，dialogue）：

- 源会话：一个已归档的施工窗，605,363 字节 → 产物 2,273 字节。
- `claude --resume` 起得来，能继续对话；拿保留段里一个可判定的事实去追问，答对了。
- 源文件跑完前后 SHA-256 不变（先整份备份过，再比的）。
- 保留的文本在源文件里逐字命中；丢掉的是 thinking、非主链与运行元数据、usage。
- 量程：那个样本的主链只识别出 1 轮，这一跑没有覆盖「真的切掉一个或多个完整轮次」那条路。

2026-10-23，macOS，Claude Code 2.1.286（v3 脚本）：

- 单元测试 25 个全过（合成数据），覆盖并行调用配对、结果逆序完成、full 剥 thinking 后链是否完整、不成对时报错、带保留原话的压缩和断链会被拒绝、二次 slim 的时间和切轮、工具结果行里夹带的人类文字、不覆盖已有文件、自检和源侧检查能拦住坏序列。
- 一位审稿人用构造的边界样本逐条复现了旧版会静默丢内容的几条路径（乱序并行、压缩保留原话、断链、夹带文字、二次 slim 时间）。修完以后，这些样本要么完整保全，要么明确拒绝写出，没有一个是「成功退出又悄悄丢了内容」。
- 本机最近 34 个真实会话，每个跑 dialogue、`--from`、full 三种 dry-run，102 次全部正常退出（只统计、不写文件）。
- 拿一份 4 轮的旧会话真切：full 模式切掉 2 轮、捞回 1 条挂在主线外的并行结果、剥掉 2 块 thinking，产物 14 行，源文件 SHA-256 前后不变。`claude -p --resume` 接上后问「前面调用过的工具都叫什么」，答对了工具名、次数、各自的参数和返回结果。

- 同一份会话 `--all` dialogue（这次没有切，只瘦身）：产物 6 行。接上后问每条人类消息的发送时间和内容，四条全对；它还主动说明前面提到的工具调用在这段对话里看不到，没法核实。这正是 dialogue 该有的样子：话都在，工具被剥掉了，它自己也知道。

我们家 runtime（完整版）：从 2026-09 起日常在用。今天写这份教程的窗口预览了一次：dialogue 模式，18.3 万 token 的窗口，剥掉 47 次工具调用、42 块 thinking，保留段约六千 token。

⇒ 现在能说的是：**在上面这些范围里可用。**长会话、切掉很多轮、压缩过的会话、其他版本和其他平台，还没有真样本覆盖。拿正式窗口跑之前，先整份备份，先用 `--list-turns` / `--dry-run` 把切点看一眼。

附录：slim.py

代码就是仓库里（或随 PDF 一起发的）`slim.py` 文件，直接用那个文件。下面的全文只供阅读；如果你手上只有这份 PDF，复制后先跑一次 `python3 -m py_compile slim.py`，报错就是缩进被排版弄坏了。

```
#!/usr/bin/env python3
"""
slim.py: 给 Claude Code 的会话做一次 slim（第三版）。

从一份旧会话文件（~/.claude/projects/<项目目录>/<会话id>.jsonl）里，
挑一个时间点，把那之前的对话整段拿掉，之后的保留下来，
写成一份新的会话文件，然后用 `claude --resume <新id>` 接着聊。

两种保留方式（--history）：
  dialogue（默认）：只带对话。人说的话、AI 说的话、语音工具（成对）、--keep-tool 指定的工具（成对）。
                    不做摘要，不改写任何一句话；只加三处标记：压缩摘要前的说明、图片占位、发送时间。
  full:             从切点那一轮起，主线上的行原样带过去（工具调用和结果、图片、附件都在），
                    只改四处：首行 parentUuid 置空并按顺序重新串链、换新 sessionId、删 message.usage、
                    剥掉 thinking。活干到一半、还要用刚才的工具结果时用它。只能配 --from。

旧文件只读，不改不删。新文件排他创建，已存在就报错，绝不覆盖。只用 Python 3.9+ 标准库。

安全提醒：不要让 Claude 用 cat / Read 直接读会话 jsonl。里面逐条存着 thinking 字段，
把它读进上下文可能触发平台拦截（[reasoning_extraction]），整轮被拒。要看有哪些轮次，就用 --list-turns。
这个脚本只在内存里处理 thinking，永远不打印 thinking 的内容。

用法：
  python3 slim.py 旧会话.jsonl --list-turns --tz Asia/Shanghai     # 先看每一轮的时间和开头
  python3 slim.py 旧会话.jsonl --from 2026-09-30T15:40:00+08:00 --dry-run
  python3 slim.py 旧会话.jsonl --from 2026-09-30T15:40:00+08:00 --tz Asia/Shanghai
  python3 slim.py 旧会话.jsonl --from 2026-09-30T15:40:00+08:00 --history full
  python3 slim.py 旧会话.jsonl --all                                # 不切，全部轮次只做瘦身
"""

import argparse
import copy
import json
import os
import re
import sys
import time
import uuid
from datetime import datetime, timezone

try:
```

```
    from zoneinfo import ZoneInfo
except ImportError:  # Python < 3.9
    ZoneInfo = None

CHAIN_TYPES = {"user", "assistant", "attachment", "system"}
THINKING_TYPES = {"thinking", "redacted_thinking"}
# Messages API 对 tool_use id 的约束
TOOL_ID_RE = re.compile(r"^[a-zA-Z0-9_-]+$")
# 语音类工具：电话里她说的话只存在于 listen 的返回里，念出口的话只存在于 speak 的输入里，
# 剥掉就两边都没了。所以默认成对保留，--no-voice 关掉。
TIME_PREFIX_RE = re.compile(r"^\[消息发送时间：[^\]\n]*\]\n")
VOICE_TOOLS = ("mcp__tts__speak", "mcp__voice__speak", "mcp__voice__listen",
               "mcp__voice__converse", "mcp__voice__hangup")


class SlimError(Exception):
    """能预见的失败：给人看的中文说明，不写文件。"""


# ---------- 读 ----------

def load_rows(path):
    rows = []
    with open(path, "r", encoding="utf-8") as f:  # 只读
        for line in f:
            if not line.strip():
                continue
            try:
                row = json.loads(line)
            except json.JSONDecodeError:
                continue  # 写了一半的行，跳过
            if isinstance(row, dict):
                rows.append(row)
    return rows


def active_chain(rows):
    """CLI 恢复时会走的那条主线：从文件里最后写入的一行，沿 parentUuid 往回走。

    rewind、重试留下的废弃分支还在文件里，但最后写入的那一行一定在主线上。
    """
    by_uuid, leaf = {}, None
    for row in rows:
        u = row.get("uuid")
        if not u or row.get("isSidechain") is True or row.get("type") not in CHAIN_TYPES:
            continue
        by_uuid.setdefault(u, row)
        leaf = row
    chain, seen, row = [], set(), leaf
    while row is not None and row["uuid"] not in seen:
        seen.add(row["uuid"])
        chain.append(row)
        parent = row.get("parentUuid")
        row = by_uuid.get(parent) if parent else None
    chain.reverse()
    return chain


def chain_break(rows, chain):
    """主线的第一行如果 parentUuid 不是 null、文件里却找不到那一行，说明会话链在这里断了。

    返回断点那一行（主线第一行），没断返回 None。压缩边界、slim 产物的第一行 parentUuid
    都是 null，是正常起点，不算断。Claude Code 恢复时遇到断链会按时间戳找一行接回去，
    slim.py 不会，所以断点之前的内容沿链走不到。
    """
    if not chain:
        return None
    head = chain[0]
    parent = head.get("parentUuid")
    if not parent:
        return None
    reachable = {r.get("uuid") for r in rows if r.get("uuid") and r.get("isSidechain") is not True
```

```
                and r.get("type") in CHAIN_TYPES}
    return head if parent not in reachable else None


def blocks(row):
    content = (row.get("message") or {}).get("content")
    return [b for b in content if isinstance(b, dict)] if isinstance(content, list) else []


def use_ids(row):
    if row.get("type") ≠ "assistant":
        return []
    return [b.get("id") for b in blocks(row) if b.get("type") == "tool_use"]


def result_ids(row):
    if row.get("type") ≠ "user":
        return []
    return [b.get("tool_use_id") for b in blocks(row) if b.get("type") == "tool_result"]


def _is_meta_tail(row):
    """CLI 恢复时会跟着并行调用一起捞回来的「附属行」：附件、非压缩边界的 system、isMeta 的 user。"""
    kind = row.get("type")
    if kind == "attachment":
        return True
    if kind == "system":
        return row.get("subtype") ≠ "compact_boundary"
    return kind == "user" and row.get("isMeta") is True and not result_ids(row)


def _belongs_to_group(row, mid):
    """主线上紧跟一组并行调用、仍算这一组的行（同一 message.id 的 assistant、结果、附属行）。"""
    kind = row.get("type")
    if kind == "assistant":
        return mid is not None and (row.get("message") or {}).get("id") == mid
    if kind == "user":
        return row.get("isMeta") is True or bool(result_ids(row))
    if kind == "attachment":
        return True
    if kind == "system":
        return row.get("subtype") ≠ "compact_boundary"
    return False


def recover_siblings(rows, chain, stats=None):
    """把并行调用留在旁支上的行接回主线，照 CLI 恢复会话时的做法（2.1.278 的 l3o）。

    一条 assistant 回复（同一个 message.id）里的多个 tool_use，CLI 是分成几行记的；
    结果完成的顺序和调用顺序不一定一样，后面的对话接在哪个结果下面，主线就只经过哪一支。
    所以按主线上每个 message.id 把同组的 assistant 行、挂在它们下面的结果行、
    以及结果后面一串纯附属行（附件、提醒）都捞回来，按文件里的先后插回这一组所在的位置。
    """
    main = [r for r in rows if r.get("uuid") and r.get("isSidechain") is not True
            and r.get("type") in CHAIN_TYPES]
    order, by_mid, results_under, children = {}, {}, {}, {}
    for n, r in enumerate(main):
        order.setdefault(r["uuid"], n)
    seen_uuid = set()
    for r in main:
        if r["uuid"] in seen_uuid:
            continue
        seen_uuid.add(r["uuid"])
        mid = (r.get("message") or {}).get("id") if r.get("type") == "assistant" else None
        if mid:
            by_mid.setdefault(mid, []).append(r)
        elif r.get("type") == "user" and r.get("parentUuid") and result_ids(r):
            results_under.setdefault(r["parentUuid"], []).append(r)
        if r.get("parentUuid"):
            children.setdefault(r["parentUuid"], []).append(r)

    taken = {r.get("uuid") for r in chain}
```

```
    pos = {r.get("uuid"): n for n, r in enumerate(chain)}

    def meta_tail(start, parent):
        out, row = [], start
        while row is not None:
            if row["uuid"] in taken or row.get("isSidechain") != parent.get("isSidechain") \
                    or not _is_meta_tail(row):
                return []
            out.append(row)
            nxt = [c for c in children.get(row["uuid"], []) if c["uuid"] not in taken]
            if len(nxt) > 1:
                return []
            row = nxt[0] if nxt else None
        return out

    spans, done, n_rows, n_tail = [], set(), 0, 0
    for r in chain:
        if r.get("type") != "assistant":
            continue
        mid = (r.get("message") or {}).get("id")
        if not mid or mid in done:
            continue
        done.add(mid)
        group = by_mid.get(mid, [r])
        got = [g for g in group if g["uuid"] not in taken]
        got += [x for g in group for x in results_under.get(g["uuid"], []) if x["uuid"] not in taken]
        for x in got:
            taken.add(x["uuid"])
        if any(use_ids(g) for g in group):
            heads = group + [x for g in group for x in results_under.get(g["uuid"], [])]
            for h in heads:
                for c in children.get(h["uuid"], []):
                    tail = meta_tail(c, h)
                    for x in tail:
                        taken.add(x["uuid"])
                    got += tail
                    n_tail += len(tail)
        if not got:
            continue
        n_rows += len(got)
        start = pos[r["uuid"]]
        end = start + 1
        for g in group:
            p = pos.get(g["uuid"])
            if p is not None and p >= end:
                end = p + 1
        while end < len(chain) and _belongs_to_group(chain[end], mid):
            end += 1
        got.sort(key=lambda x: order.get(x["uuid"], len(main)))
        spans.append([start, end, got])

    if stats is not None:
        stats["recovered_sibling_rows"] = n_rows
    if not spans:
        return list(chain)
    spans.sort(key=lambda s: s[0])
    merged = []
    for s in spans:
        if merged and s[0] < merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], s[1])
            merged[-1][2] = sorted(merged[-1][2] + s[2], key=lambda x: order.get(x["uuid"], len(main)))
        else:
            merged.append(s)
    out, cursor = [], 0
    for start, end, got in merged:
        out.extend(chain[cursor:start + 1])
        k = 0
        for row in chain[start + 1:end]:
            here = order.get(row["uuid"], len(main))
            while k < len(got) and order.get(got[k]["uuid"], len(main)) < here:
                out.append(got[k])
                k += 1
```

```
            out.append(row)
        out.extend(got[k:])
        cursor = end
    out.extend(chain[cursor:])
    return out


def preserved_compactions(rows):
    """带 preservedSegment / preservedMessages 的压缩边界：[(边界行, anchor uuid, 保留段 uuid 列表)]。

    这种压缩会把压缩前的一小段原话留着，CLI 恢复时把它接到摘要后面
    （边界 → 摘要 → 保留段 → 之后的对话）。这段接线不在 parentUuid 里，沿链回溯走不到。
    """
    by_uuid = {}
    for r in rows:
        if r.get("uuid"):
            by_uuid.setdefault(r["uuid"], r)
    found = []
    for r in rows:
        if r.get("isSidechain") is True or r.get("type") != "system" \
                or r.get("subtype") != "compact_boundary":
            continue
        meta = r.get("compactMetadata") or {}
        seg, msgs = meta.get("preservedSegment"), meta.get("preservedMessages")
        if not seg and not msgs:
            continue
        anchor = (msgs or {}).get("anchorUuid") or (seg or {}).get("anchorUuid")
        uuids = list((msgs or {}).get("uuids") or [])
        if not uuids and isinstance(seg, dict) and seg.get("tailUuid"):
            row, guard = by_uuid.get(seg["tailUuid"]), 0
            while row is not None and guard < 100000:
                uuids.append(row["uuid"])
                if row["uuid"] == seg.get("headUuid"):
                    break
                row, guard = by_uuid.get(row.get("parentUuid")), guard + 1
            uuids.reverse()
        found.append((r, anchor, uuids))
    return found


def check_preserved_compaction(rows, chain, turns, first, tz=None):
    """保留段会被切进新会话、却又没按 CLI 的方式接在摘要后面时，拒绝写出。"""
    pos = {r.get("uuid"): n for n, r in enumerate(chain)}
    kept_from = turns[first]["begin"]
    for boundary, anchor, uuids in preserved_compactions(rows):
        marks = [boundary.get("uuid"), anchor] + uuids
        idx = [pos[m] for m in marks if m in pos]
        if not idx or max(idx) < kept_from:
            continue  # 这次压缩整个落在切掉的那一段里（或者是已经不在主线上的旧压缩）
        linked = (all(m in pos for m in marks) and pos[boundary["uuid"]] < pos[anchor]
                  and all(pos[anchor] < pos[u] for u in uuids))
        if linked:
            continue  # 保留段已经实实在在接在摘要后面，主线上一样不少
        later = next((t for t in turns if t["begin"] > max(idx)), None)
        hint = (f"要 slim 的话，把 --from 设成 {iso_label(chain[later['start']].get('timestamp'), tz) or '压缩之后那
                f" 或更晚（只保留这次压缩之后的轮次）"
                if later else "这次压缩之后还没有完整的轮次，先在 Claude Code 里接着聊几轮再 slim")
        raise SlimError(
            "这份会话被 Claude Code 压缩过，而且压缩时留了一段压缩前的原话"
            "（compactMetadata.preservedSegment / preservedMessages）。"
            "Claude Code 恢复时会把它接成 压缩边界 → 摘要 → 保留的原话 → 之后的对话，"
            "这段接线不在 parentUuid 里；slim.py 还不会照着接，硬切会把摘要或保留的原话静默丢掉，"
            f"所以这次不写文件。{hint}。")


def sibling_results(rows, chain):
    """并行工具调用的结果，有一部分不在主线上。

    Claude Code 记并行调用时，每个结果行挂在自己那条调用行下面，
    从最后一行往回走只会经过最后一个结果，前面几个结果在旁支上（CLI 恢复时自己会捞回来）。
    这里同样捞回来：tool_use_id → 那条结果行（不在主线、不是子 agent）。
    recover_siblings 之后大多已经在主线上，这里是兜底。
```

```
    """
    on_chain = {r.get("uuid") for r in chain}
    found = {}
    for row in rows:
        if row.get("isSidechain") is True or row.get("uuid") in on_chain:
            continue
        for rid in result_ids(row):
            if rid:
                found.setdefault(rid, row)
    return found


def is_prompt(row):
    """一轮的开头：真正由人输入的那条消息（不是工具结果、系统提醒、压缩摘要）。"""
    if row.get("type") != "user" or row.get("isMeta") or row.get("isCompactSummary"):
        return False
    content = (row.get("message") or {}).get("content")
    if isinstance(content, str):
        return bool(content.strip())
    types = [b.get("type") for b in blocks(row)]
    return "tool_result" not in types and any(t in ("text", "image") for t in types)


def parse_time(value):
    if not value:
        return None
    try:
        t = datetime.fromisoformat(str(value).replace("Z", "+00:00"))
    except ValueError:
        return None
    return t if t.tzinfo else None  # 没有时区的时间戳不可信，当作缺失


def split_turns(chain):
    starts = [i for i, row in enumerate(chain) if is_prompt(row)]
    turns = []
    for n, start in enumerate(starts):
        begin = 0 if n == 0 else start  # 第一轮之前的东西（比如压缩摘要、上次 slim 的头）算进第一轮
        end = starts[n + 1] if n + 1 < len(starts) else len(chain)
        times = [t for t in (parse_time(r.get("timestamp")) for r in chain[begin:end]) if t]
        turns.append({"begin": begin, "start": start, "end": end,
                      "started": parse_time(chain[start].get("timestamp")),
                      "ended": max(times) if times else None})
    return turns


def prompt_text(row):
    """人说的话的文字。只取 text，图片换占位；永远不碰 thinking。"""
    content = (row.get("message") or {}).get("content")
    if isinstance(content, str):
        return content
    parts = []
    for b in blocks(row):
        if b.get("type") == "text":
            parts.append(b.get("text", ""))
        elif b.get("type") == "image":
            parts.append("[图片，slim 时省略]")
    return "\n".join(p for p in parts if p)


# ---------- 时区 ----------

def local_zone_name():
    """本机时区的名字，尽量拿到 Asia/Shanghai 这种形式。"""
    tz_env = os.environ.get("TZ", "").lstrip(":")
    if tz_env and "/" in tz_env:
        return tz_env
    try:
        target = os.path.realpath("/etc/localtime")
        if "zoneinfo/" in target:
            return target.split("zoneinfo/", 1)[1]
    except OSError:
```

```
        pass
    try:
        with open("/etc/timezone", encoding="utf-8") as f:  # 部分 Linux
            name = f.read().strip()
            if name:
                return name
    except OSError:
        pass
    return None


def resolve_tz(name):
    """返回 (tzinfo 或 None, 给人看的说明)。None 表示用本机时区 (astimezone())。"""
    if name:
        if ZoneInfo is None:
            raise SlimError("这个 Python 没有 zoneinfo (需要 3.9+), 不能用 --tz。")
        try:
            return ZoneInfo(name), None
        except Exception:
            raise SlimError(f"认不出时区 {name}。写成 Asia/Shanghai 这种形式; "
                            "Windows 上可能要先 pip install tzdata。")
    zone = local_zone_name()
    offset = datetime.now().astimezone().strftime("%z")
    offset = offset[:3] + ":" + offset[3:] if offset else "?"
    if zone and ZoneInfo is not None:
        try:
            return ZoneInfo(zone), f"使用本机时区 {zone} (现在是 UTC{offset}); 要换就加 --tz"
        except Exception:
            pass
    abbr = time.tzname[0] if time.tzname else "?"
    return None, f"使用本机时区 {abbr} (现在是 UTC{offset}); 要换就加 --tz"


def to_local(ts, tz):
    t = parse_time(ts)
    if not t:
        return None
    return t.astimezone(tz) if tz else t.astimezone()


def short_label(ts, tz):
    t = to_local(ts, tz)
    return t.strftime("%Y-%m-%d %H:%M") if t else None


def iso_label(ts, tz):
    t = to_local(ts, tz)
    return t.isoformat(timespec="seconds") if t else None


# ---------- dialogue: 投影成对话 ----------

def interjection_texts(row):
    """工具结果行里夹带的文字块 (人在工具跑的时候插的话)。<system-reminder> 开头的是系统注入, 不算。"""
    if row.get("type") != "user" or row.get("isMeta") or not result_ids(row):
        return []
    out = []
    for b in blocks(row):
        if b.get("type") != "text":
            continue
        text = b.get("text") or ""
        if not text.strip() or text.lstrip().startswith("<system-reminder>"):
            continue
        out.append(text)
    return out


def project_dialogue(kept, keep_tools, extra_results, add_time, tz, stats):
    """把保留段投影成对话事件: 人说的话、AI 说的话、要保留的工具 (调用和结果一起)。

    工具以「一对」为单位出事件: 遇到 tool_use 时把它的结果一起带上,
    结果行后面再遇到就跳过。并行调用因此也是 调用1→结果1→调用2→结果2。
```

```
    结果行里夹带的人类文字单独出一条人说的话：结果属于保留的工具对时紧跟在那一对后面，
    否则留在结果行原来的位置。
    """
    results = {}  # tool_use_id → (结果块, toolUseResult, 时间戳)
    result_row = {}  # tool_use_id → 结果所在的那一行
    for row in kept:
        for b in blocks(row):
            if row.get("type") == "user" and b.get("type") == "tool_result":
                results.setdefault(b.get("tool_use_id"),
                                   (b, row.get("toolUseResult"), row.get("timestamp")))
                result_row.setdefault(b.get("tool_use_id"), row)
    for rid, row in extra_results.items():
        for b in blocks(row):
            if b.get("type") == "tool_result" and b.get("tool_use_id") == rid:
                results.setdefault(rid, (b, row.get("toolUseResult"), row.get("timestamp")))
                result_row.setdefault(rid, row)

    def stamp(text, ts):
        if not add_time or TIME_PREFIX_RE.match(text):
            return text  # 不加时间，或上一次 slim 已经加过，不再叠一层
        label = iso_label(ts, tz)
        if label:
            return f"[消息发送时间：{label}]\n{text}"
        stats["no_time_prompts"] += 1
        return text

    events = []  # (kind, payload, timestamp)
    spoken = set()  # 夹带文字已经出过事件的结果行
    # 夹带文字的结果行里有几个结果属于会保留的工具对：数到 0 时文字紧跟最后那一对出
    kept_names = {b.get("id"): b.get("name") for r in kept if r.get("type") == "assistant"
                  for b in blocks(r) if b.get("type") == "tool_use"}
    waiting = {}
    for row in list(kept) + list(extra_results.values()):
        if id(row) in waiting or not interjection_texts(row):
            continue
        n = sum(1 for i in result_ids(row) if kept_names.get(i) in keep_tools)
        if n:
            waiting[id(row)] = n

    def emit_interjection(row):
        if id(row) in spoken:
            return
        spoken.add(id(row))
        texts = interjection_texts(row)
        if texts:
            stats["interjections"] = stats.get("interjections", 0) + len(texts)
            events.append(("user", stamp("\n".join(texts), row.get("timestamp")), row.get("timestamp")))

    for row in kept:
        ts = row.get("timestamp")
        kind = row.get("type")
        if kind == "user":
            if row.get("isCompactSummary"):
                content = (row.get("message") or {}).get("content")
                text = content if isinstance(content, str) else prompt_text(row)
                if text.strip():
                    note = "[这之前的对话被 Claude Code 自动压缩过，下面是压缩摘要]\n"
                    events.append(("user", note + text, ts))
            elif is_prompt(row):
                text = prompt_text(row)
                if text.startswith(("<command-", "<local-command-")):
                    continue  # 斜杠命令和它的输出，不是对话
                events.append(("user", stamp(text, ts), ts))
            elif result_ids(row):
                # 工具结果行：结果已经跟着它的调用出过了；夹带的人类文字没跟着出过的，在这里出
                emit_interjection(row)
        elif kind == "assistant":
            for b in blocks(row):
                t = b.get("type")
                if t == "text" and b.get("text", "").strip():
                    events.append(("text", b["text"], ts))
                elif t == "tool_use":
```

```
                    hit = results.get(b.get("id"))
                    if b.get("name") in keep_tools and hit is not None:
                        events.append(("tool_pair", {"use": b, "result": hit}, ts))
                        rrow = result_row.get(b.get("id"))
                        if rrow is not None and id(rrow) in waiting:
                            waiting[id(rrow)] -= 1
                            if waiting[id(rrow)] <= 0:  # 这一行里最后一个保留的工具对出完了
                                emit_interjection(rrow)
                    # 其他工具、没有结果的调用：丢掉
                # thinking、图片之外的其他块：丢掉
    return events


def build_dialogue_rows(events, session_id, cwd, model, stats):
    rows, parent, pending, pending_ts = [], None, [], None
    used_ids = set()
    stats.setdefault("id_map", [])
    known = [ts for _, _, ts in events if parse_time(ts)]
    fallback = known[0] if known else datetime.now(timezone.utc).isoformat()

    def fix_ts(ts):
        nonlocal fallback
        if parse_time(ts):
            fallback = ts
            return ts
        stats["no_time_rows"] += 1  # 缺时间戳：借用前一行的时间，并计数警告
        return fallback

    def push(kind, message, ts, extra=None):
        nonlocal parent
        u = str(uuid.uuid4())
        row = {"parentUuid": parent, "isSidechain": False, "type": kind, "uuid": u,
               "timestamp": fix_ts(ts), "sessionId": session_id, "cwd": cwd, "message": message}
        if extra:
            row.update(extra)
        rows.append(row)
        parent = u

    def add_pending(block, ts):
        # 攒着的 assistant 内容记它自己最后一个事件的时间，不借下一条 user 的时间
        nonlocal pending_ts
        pending.append(block)
        if parse_time(ts) or pending_ts is None:
            pending_ts = ts

    def flush():
        nonlocal pending, pending_ts
        if pending:
            message = {"id": "msg_" + uuid.uuid4().hex, "type": "message",
                       "role": "assistant", "model": model, "content": pending}
            push("assistant", message, pending_ts)
            pending, pending_ts = [], None

    def tool_id(raw):
        tid = str(raw or "")
        if not TOOL_ID_RE.match(tid) or tid in used_ids:
            tid = "toolu_vslim_" + uuid.uuid4().hex
            stats["renamed_tool_ids"] += 1
        used_ids.add(tid)
        stats["id_map"].append((raw, tid))
        return tid

    for kind, payload, ts in events:
        if kind == "user":
            flush()
            push("user", {"role": "user", "content": payload}, ts)
        elif kind == "text":
            add_pending({"type": "text", "text": payload}, ts)
        elif kind == "tool_pair":
            use = payload["use"]
            block, tool_use_result, result_ts = payload["result"]
            tid = tool_id(use.get("id"))
```

```
            add_pending({"type": "tool_use", "id": tid, "name": use.get("name"),
                         "input": use.get("input") or {}}, ts)
            flush()  # 调用结束这条 assistant 消息，下一行紧跟它的结果
            result = {"type": "tool_result", "tool_use_id": tid,
                      "content": block.get("content", "")}
            if block.get("is_error"):
                result["is_error"] = True
            extra = {"toolUseResult": tool_use_result} if tool_use_result is not None else None
            push("user", {"role": "user", "content": [result]}, result_ts or ts, extra)
    flush()
    return rows


# ---------- full：原行带过去 ----------

def build_full_rows(kept, extra_results, session_id, cwd_override, stats):
    """保留段原行深拷贝。只改：剥 thinking、首行 parentUuid=null 并重新串链、
    换 sessionId、删 message.usage（以及给了 --cwd 时的 cwd）。"""
    rows = [copy.deepcopy(r) for r in kept]
    on_chain = {r.get("uuid") for r in rows}
    kept_uses = {i for r in rows for i in use_ids(r)}
    kept_answered = {i for r in rows for i in result_ids(r)}
    extras = []
    for rid, row in extra_results.items():
        # 只捞保留段里有调用、却还没有结果的那几条
        if rid not in kept_uses or rid in kept_answered:
            continue
        if row.get("uuid") not in on_chain:
            on_chain.add(row.get("uuid"))
            extras.append(copy.deepcopy(row))
    stats["recovered_results"] = len(extras)
    rows.extend(extras)

    # 1. 剥 thinking；剥完空了的 assistant 行整行删掉
    cleaned = []
    for row in rows:
        content = (row.get("message") or {}).get("content")
        if row.get("type") == "assistant" and isinstance(content, list):
            keep = [b for b in content if not (isinstance(b, dict) and b.get("type") in THINKING_TYPES)]
            stripped = len(content) - len(keep)
            if stripped:
                stats["thinking"] += stripped
                row["message"]["content"] = keep
                if not keep:
                    stats["dropped_rows"] += 1
                    continue
        cleaned.append(row)
    rows = cleaned

    # 2. 配对检查。最后一轮里还没返回的调用（比如正在跑 slim 的这一次）去掉，其余不成对就拒绝
    uses = {i for r in rows for i in use_ids(r)}
    answered = {i for r in rows for i in result_ids(r)}
    orphans = [i for i in answered if i not in uses]
    unanswered = [i for i in uses if i not in answered]
    if unanswered:
        last_prompt = max((n for n, r in enumerate(rows) if is_prompt(r)), default=-1)
        final_ids = set()
        last_mid = None
        for n in range(len(rows) - 1, -1, -1):  # 最后一条 assistant 消息（同一个 message.id）
            if rows[n].get("type") == "assistant":
                last_mid = (rows[n].get("message") or {}).get("id")
                break
        for n, r in enumerate(rows):
            if n > last_prompt and r.get("type") == "assistant" and \
                    (r.get("message") or {}).get("id") == last_mid:
                final_ids.update(use_ids(r))
        in_flight = [i for i in unanswered if i in final_ids]
        unanswered = [i for i in unanswered if i not in final_ids]
        if in_flight and not unanswered and not orphans:
            stats["dropped_tools"] += len(in_flight)
            drop = set(in_flight)
            kept_rows = []
```

```
            for r in rows:
                if r.get("type") == "assistant" and drop & set(use_ids(r)):
                    r["message"]["content"] = [b for b in blocks(r) if not (
                        b.get("type") == "tool_use" and b.get("id") in drop)]
                    if not r["message"]["content"]:
                        stats["dropped_rows"] += 1
                        continue
                kept_rows.append(r)
            rows = kept_rows
    if unanswered or orphans:
        raise SlimError(f"保留段里的工具调用不成对：{len(unanswered)} 个调用没有结果，"
                        f"{len(orphans)} 个结果找不到调用。full 模式不能只带一半（API 会拒绝）。"
                        "把切点往前或往后挪一轮，或者改用 --history dialogue。")

    # 3. 每个调用行后面紧跟它的结果行（并行调用：调用1→结果1→调用2→结果2，CLI 自己也这样记）
    by_result = {}
    for r in rows:
        for rid in result_ids(r):
            by_result.setdefault(rid, r)
    result_rows = {id(r) for r in by_result.values()}
    ordered, placed = [], set()
    for r in rows:
        if id(r) in result_rows:
            continue
        ordered.append(r)
        for uid in use_ids(r):
            rr = by_result.get(uid)
            if rr is not None and id(rr) not in placed:
                placed.add(id(rr))
                ordered.append(rr)
    rows = ordered

    # 4. 重新串链、换 sessionId、删 usage
    parent = None
    for r in rows:
        r["parentUuid"] = parent
        r["sessionId"] = session_id
        if cwd_override:
            r["cwd"] = cwd_override
        if isinstance(r.get("message"), dict):
            r["message"].pop("usage", None)
        if r.get("type") in ("user", "assistant") and not parse_time(r.get("timestamp")):
            stats["no_time_rows"] += 1
        parent = r.get("uuid")
    return rows


# ---------- 自检 ----------

def source_preservation(kept, extra, out_rows, keep_tools, id_map):
    """源侧保全检查：保留段（源主线 + 捞回的旁支）里应该带过去的 tool_use，产物里必须都有。

    「应该带过去」只按源来定，不看构建过程：有结果（在保留段里或旁支上）的调用；
    dialogue 再限定在要保留的工具里（keep_tools），full 是全部。
    没结果的调用（比如正在跑 slim 的这一次）两种模式都会去掉，不算。
    返回（应保留的源 id 列表，产物里找不到的源 id 列表）。
    """
    answered = {i for r in kept for i in result_ids(r)} | set(extra)
    want = []
    for r in kept:
        for b in blocks(r):
            if r.get("type") != "assistant" or b.get("type") != "tool_use":
                continue
            if b.get("id") not in answered:
                continue
            if keep_tools is not None and b.get("name") not in keep_tools:
                continue
            want.append(b.get("id"))
    out_ids = {i for r in out_rows for i in use_ids(r)}
    out_results = {i for r in out_rows for i in result_ids(r)}
    if keep_tools is None:
        produced = {i for i in out_ids if i in out_results}
```

```
    else:  # dialogue 可能给不合法、重复的 id 换了新名字
        produced = {raw for raw, new in id_map if new in out_ids and new in out_results}
    missing = [i for i in dict.fromkeys(want) if i not in produced]
    return want, missing


def validate(rows, session_id):
    """写文件之前的自检。返回问题列表（只含行号和 id，不含任何内容）；空列表才写。"""
    problems = []
    if not rows:
        return ["新会话一行都没有"]
    seen = set()
    for n, r in enumerate(rows):
        u = r.get("uuid")
        if not u:
            problems.append(f"第 {n + 1} 行没有 uuid")
        elif u in seen:
            problems.append(f"第 {n + 1} 行 uuid 重复：{u}")
        seen.add(u)
        want = None if n == 0 else rows[n - 1].get("uuid")
        if r.get("parentUuid") != want:
            problems.append(f"第 {n + 1} 行 parentUuid 没有指向上一行" if n else
                            "第 1 行 parentUuid 不是 null")
        if r.get("sessionId") != session_id:
            problems.append(f"第 {n + 1} 行 sessionId 不是新 id")
        message = r.get("message")
        if isinstance(message, dict) and "usage" in message:
            problems.append(f"第 {n + 1} 行还带着 message.usage")
        for b in blocks(r):
            if b.get("type") in THINKING_TYPES:
                problems.append(f"第 {n + 1} 行还有 {b.get('type')} 块")

    tool_seen = set()
    for n, r in enumerate(rows):
        ids = use_ids(r)
        for i in ids:
            if not isinstance(i, str) or not TOOL_ID_RE.match(i):
                problems.append(f"第 {n + 1} 行 tool_use.id 不合法：{i!r}")
            elif i in tool_seen:
                problems.append(f"第 {n + 1} 行 tool_use.id 重复：{i}")
            tool_seen.add(i)
        if ids:
            # 下一行（连续几行结果行）必须含它的 tool_result
            got, m = set(), n + 1
            while m < len(rows) and result_ids(rows[m]):
                got.update(result_ids(rows[m]))
                m += 1
            for i in ids:
                if i not in got:
                    problems.append(f"第 {n + 1} 行的 tool_use {i} 下一行不是它的 tool_result")
        rids = result_ids(r)
        if rids:
            # 往前跨过连续的结果行，第一条非结果行必须是含对应 tool_use 的那行
            m = n - 1
            while m >= 0 and result_ids(rows[m]):
                m -= 1
            before = set(use_ids(rows[m])) if m >= 0 else set()
            for i in rids:
                if i not in before:
                    problems.append(f"第 {n + 1} 行的 tool_result {i} 前一行没有对应的 tool_use")
    return problems


# ---------- 写 ----------

def project_dir_for(cwd):
    slug = "".join(c if (c.isalnum() and c.isascii()) or c in ".-" else "-" for c in cwd)
    return os.path.join(os.path.expanduser("~"), ".claude", "projects", slug)


def serialize(rows):
    return "".join(json.dumps(r, ensure_ascii=False) + "\n" for r in rows).encode("utf-8")
```

```
def write_session(path, data):
    """排他创建：已存在就抛 FileExistsError，绝不覆盖。写完 fsync 文件和目录。"""
    fd = os.open(path, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    try:
        view = memoryview(data)
        while view:
            n = os.write(fd, view)
            view = view[n:]
        os.fsync(fd)
    finally:
        os.close(fd)
    try:
        dfd = os.open(os.path.dirname(path) or ".", os.O_RDONLY)
        try:
            os.fsync(dfd)
        finally:
            os.close(dfd)
    except OSError:
        pass  # 有的文件系统不支持对目录 fsync，文件本身已经落盘


# ---------- 主流程 ----------

def parse_args(argv=None):
    ap = argparse.ArgumentParser(
        description="给 Claude Code 会话做一次 slim：切掉某个时间点之前的轮次，保留之后的原话")
    ap.add_argument("source", help="旧会话文件 .jsonl（只读）")
    ap.add_argument("--from", dest="retain_from",
                    help="保留这个时间点之后结束的完整轮次，必须带时区，如 2026-09-30T15:40:00+08:00")
    ap.add_argument("--all", action="store_true",
                    help="不切，全部轮次都保留，只做 dialogue 瘦身（与 --from 二选一）")
    ap.add_argument("--history", choices=("dialogue", "full"), default="dialogue",
                    help="dialogue（默认）只带对话；full 把保留段原行带过去（只能配 --from）")
    ap.add_argument("--keep-tool", action="append", default=[],
                    help="dialogue 模式下额外成对保留的工具名，可以写多次")
    ap.add_argument("--no-voice", action="store_true",
                    help="不默认保留语音工具（" + "、".join(VOICE_TOOLS) + "）")
    ap.add_argument("--no-time", action="store_true", help="不在人说的话前面加发送时间")
    ap.add_argument("--tz", help="显示和发送时间用的时区，例如 Asia/Shanghai；默认用本机时区")
    ap.add_argument("--cwd", help="新会话所属的项目目录；默认和旧会话相同")
    ap.add_argument("--list-turns", action="store_true",
                    help="列出每一轮的时间和人说的话的开头，不写文件")
    ap.add_argument("--dry-run", action="store_true", help="只统计，不写文件")
    return ap.parse_args(argv)


def run(args):
    tz, tz_note = resolve_tz(args.tz)
    if tz_note:
        print(tz_note)

    rows = load_rows(args.source)
    recover_stats = {}
    chain = recover_siblings(rows, active_chain(rows), recover_stats)
    turns = split_turns(chain)
    if not turns:
        raise SlimError("这份文件里找不到对话轮次。")

    broken = chain_break(rows, chain)
    broken_at = parse_time(broken.get("timestamp")) if broken else None
    broken_label = (iso_label(broken.get("timestamp"), tz) or "（时间未知）") if broken else None

    if args.list_turns:
        if broken:
            print(f"警告：会话链在 {broken_label} 附近断开（那一行的 parentUuid 在文件里找不到），"
                  "下面只列得出断点之后的轮次；断点之前的内容 slim.py 沿链走不到。")
        for n, t in enumerate(turns):
            head = prompt_text(chain[t["start"]]).replace("\n", " ")[:50]
            label = short_label(chain[t["start"]].get("timestamp"), tz) or "?"
            print(f"{n:4d}  {label:16s}  {head}")
```

```
        return 0

    if args.all and args.retain_from:
        raise SlimError("--all 和 --from 只能二选一：--all 是全部保留，--from 是从某个时间点切。")
    if args.all and args.history == "full":
        raise SlimError("--all 不能配 --history full：那等于把要离开的整个 context 原样搬过去，"
                        "什么都没省。全部保留就用 --all（dialogue）；要带工具结果就用 --from 切掉前面一段。")
    if not args.all and not args.retain_from:
        raise SlimError("需要 --from（带时区的时间点）或者 --all。先用 --list-turns 看看每一轮的时间。")

    if args.all:
        first = 0
    else:
        cut_at = parse_time(args.retain_from)
        if not cut_at:
            raise SlimError("--from 必须是带时区的时间，例如 2026-09-30T15:40:00+08:00")
        if cut_at > datetime.now(timezone.utc):
            raise SlimError("--from 在未来。")
        first = next((i for i, t in enumerate(turns) if t["ended"] and t["ended"] >= cut_at), None)
        if first is None:
            raise SlimError("这个时间点之后没有完整的轮次。")
    check_preserved_compaction(rows, chain, turns, first, tz)
    if broken and not (not args.all and broken_at and cut_at > broken_at):
        raise SlimError(
            f"会话链在 {broken_label} 附近断开：那一行的 parentUuid 指向一行文件里找不到的消息。"
            "Claude Code 恢复时会按时间戳把断点接回去，slim.py 不会；硬切会把断点之前的内容静默丢掉，"
            f"所以这次不写文件。如果断点之前的内容本来就不要，把 --from 设成晚于 {broken_label} 的时间再试。")
    kept = chain[turns[first]["begin"]:]
    extra = sibling_results(rows, chain)

    models = [r["message"].get("model") for r in chain
              if r.get("type") == "assistant" and isinstance(r.get("message"), dict)]
    models = [m for m in models if m and m != "<synthetic>"]
    model = models[-1] if models else "claude"
    cwd = args.cwd or next((r.get("cwd") for r in reversed(chain) if r.get("cwd")), None)
    if not cwd:
        raise SlimError("旧会话里没有 cwd，请用 --cwd 指定项目目录。")
    if args.cwd and not os.path.isabs(args.cwd):
        raise SlimError("--cwd 必须是绝对路径。")

    stats = {"no_time_prompts": 0, "no_time_rows": 0, "renamed_tool_ids": 0, "thinking": 0,
             "dropped_rows": 0, "dropped_tools": 0, "recovered_results": 0, "id_map": []}
    kept_uses = sum(1 for r in kept for b in blocks(r) if b.get("type") == "tool_use")
    kept_thinking = sum(1 for r in kept for b in blocks(r) if b.get("type") in THINKING_TYPES)
    kept_images = sum(1 for r in kept for b in blocks(r) if b.get("type") == "image")

    new_id = str(uuid.uuid4())
    keep_tools = None
    if args.history == "full":
        if args.keep_tool or args.no_voice:
            print("提示：full 模式保留全部工具，--keep-tool / --no-voice 不起作用。")
        out_rows = build_full_rows(kept, extra, new_id, args.cwd, stats)
    else:
        keep_tools = set(args.keep_tool)
        if not args.no_voice:
            keep_tools.update(VOICE_TOOLS)
        events = project_dialogue(kept, keep_tools, extra, not args.no_time, tz, stats)
        out_rows = build_dialogue_rows(events, new_id, cwd, model, stats)
        stats["dropped_tools"] = kept_uses - sum(
            1 for r in out_rows for b in blocks(r) if b.get("type") == "tool_use")
        stats["thinking"] = kept_thinking

    problems = validate(out_rows, new_id)
    want, missing_ids = source_preservation(kept, extra, out_rows, keep_tools, stats["id_map"])
    data = serialize(out_rows)

    # ---- 统计 ----
    first_turn, last_turn = turns[first], turns[-1]
    mode = "全部保留（--all）" if args.all else f"从 --from {args.retain_from} 切"
    print(f"模式：{args.history}，{mode}")
    print(f"总轮次 {len(turns)}，切掉 {first} 轮，保留第 {first} 轮起的 {len(turns) - first} 轮")
    print(f"保留段：第一轮开始于 {short_label(chain[first_turn['start']].get('timestamp'), tz) or '?'}，"
```

```
            f"最后一轮结束于 {short_label(last_turn['ended'].isoformat() if last_turn['ended'] else None, tz) or '?'
    if args.history == "full":
        print(f"保留段里：工具调用 {kept_uses - stats['dropped_tools']} 次带过去"
              f"（丢掉还没返回的 {stats['dropped_tools']} 次，捞回并行调用的旁支结果 "
              f"{stats['recovered_results']} 条），剥掉 thinking {stats['thinking']} 块"
              f"（因此整行删掉 {stats['dropped_rows']} 行），图片 {kept_images} 张原样带过去")
    else:
        print(f"保留段里：丢掉工具调用 {stats['dropped_tools']} 次，thinking {stats['thinking']} 块，"
              f"图片 {kept_images} 张（只留占位），"
              f"工具结果行里夹带的人类文字 {stats.get('interjections', 0)} 条（单独带过去）")
    print(f"源保留段工具调用 {len(want)} / 产物 {len(want) - len(missing_ids)}"
          f"（源主线含捞回的并行旁支 {recover_stats.get('recovered_sibling_rows', 0)} 行；"
          + ("full 数全部有结果的调用" if keep_tools is None else "dialogue 只数要保留且有结果的工具")
          + "）")
    print(f"新会话 {len(out_rows)} 行，产物 {len(data)} 字节，"
          f"粗估约 {len(data) // 3} token（字节÷3 的粗估，只看量级）")
    if broken:
        print(f"警告：会话链在 {broken_label} 附近断开；切点在断点之后，断点之前的内容本来就切掉，照常继续")
    if stats["renamed_tool_ids"]:
        print(f"注意：{stats['renamed_tool_ids']} 个 tool_use id 不合法或重复，已换成新 id")
    missing = stats["no_time_prompts"] + stats["no_time_rows"]
    if missing:
        print(f"警告：保留的行里有 {missing} 处缺时间戳"
              f"（{stats['no_time_prompts']} 条人说的话没加上发送时间，"
              f"{stats['no_time_rows']} 行借用了前一行的时间或保持原样）")

    if missing_ids:
        raise SlimError(f"源侧保全检查没通过：保留段里有 {len(missing_ids)} 个应该带过去的工具调用"
                        "在新会话里找不到（产物自检通过不代表内容没丢）。不写文件。"
                        "这是 slim.py 的缺陷，请把这份会话的形状（不含正文）报给作者。")
    if problems:
        print("自检没通过，不写文件：", file=sys.stderr)
        for p in problems[:20]:
            print("  - " + p, file=sys.stderr)
        if len(problems) > 20:
            print(f"  ……还有 {len(problems) - 20} 条", file=sys.stderr)
        return 2
    print("自检通过：工具调用都紧跟结果、uuid 唯一、单链、无 usage、无 thinking")

    if args.dry_run:
        print("dry-run：没有写文件。")
        return 0

    # 默认写在旧文件旁边：同一个项目目录，claude --resume 才找得到
    out_dir = project_dir_for(cwd) if args.cwd else os.path.dirname(os.path.abspath(args.source))
    os.makedirs(out_dir, mode=0o700, exist_ok=True)
    out_path = os.path.join(out_dir, new_id + ".jsonl")
    if os.path.abspath(out_path) == os.path.abspath(args.source):
        raise SlimError("新文件路径和旧文件相同，停下。")
    try:
        write_session(out_path, data)
    except FileExistsError:
        raise SlimError(f"{out_path} 已经存在，不覆盖。再跑一次会换一个新 id。")
    print(f"已写入 {out_path}")
    print(f"新 session id：{new_id}")
    print(f"接着聊：cd {cwd} && claude --resume {new_id}")
    print("有自己的 bridge 管 session_id 的，把记着的 id 换成这个即可")
    return 0


def main(argv=None):
    args = parse_args(argv)
    try:
        return run(args)
    except SlimError as e:
        print(str(e), file=sys.stderr)
        return 1


if __name__ == "__main__":
    sys.exit(main())
```